

Project Report : CS 7643

Jun Ming, Chen
Georgia Institute of Technology
North Avenue, Atlanta, GA 30332
jchen3187@gatech.edu

Zi Yi, Ker
Georgia Institute of Technology
North Avenue, Atlanta, GA 30332
zker3@gatech.edu

Abstract

This study presents a comprehensive empirical evaluation of Kolmogorov-Arnold Networks (KANs) for financial time series forecasting. We investigate two KAN variants - Temporal KAN (T-KAN) and Multivariate Temporal KAN (MT-KAN) - comparing their performance against traditional architectures including Multi-Layer Perceptrons (MLPs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) networks. Using daily S&P 500 volatility data and macroeconomic indicators from the Federal Reserve Economic Data (FRED), we assess these models across multiple forecast horizons (1-day, 1-week, and 1-month) using standard error metrics and computational efficiency measures. Our results reveal that while KANs demonstrate promising interpretability through their learnable activation functions, they face significant practical challenges including training times much longer than equivalent MLPs and unexpected parameter inefficiency. MT-KAN achieves superior accuracy for 1-day forecasts but underperforms simpler architectures at longer horizons. Additionally, we find that KAN's theoretical advantages in parameter efficiency do not consistently translate to practical implementations, with our MT-KAN requiring substantially more parameters than initially hypothesized to achieve competitive performance. These findings suggest that while KAN represents an important theoretical advancement, further research is needed to address its computational limitations before widespread adoption in time series applications.

1. Introduction

This study examines the potential of Kolmogorov-Arnold Networks (KANs) in time series forecasting. KANs represent a fundamental shift from traditional neural architectures through their unique approach of replacing fixed activation functions with trainable univariate functions along network edges [4]. Mathematically, a KAN can be

expressed as:

$$KAN(x) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_1 \circ \Phi_0)x \quad (1)$$

where each layer Φ_l consists of learnable univariate functions $\phi_{q,p}$ parametrized as splines, Inspired by the Kolmogorov-Arnold representation theorem [2, 3, 1].

Recent developments have led to two specialized KAN variants for temporal data: Temporal KAN (T-KAN) and Multivariate Temporal KAN (MT-KAN) [8]. T-KAN employs a sliding window approach for univariate series, with output at time $t + T$ expressed as:

$$\hat{S}_{t+T} = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^h \varphi_{q,p}(S_{t-h+p}) \right) \quad (2)$$

MT-KAN extends this to multivariate scenarios by incorporating cross-variable interactions:

$$\hat{S}_{t+T} = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^h \sum_{k=1}^m \varphi_{q,p,k}(S_{t-h+p,k}) \right) \quad (3)$$

This design completely eliminates linear weights, potentially offering better parameter efficiency - KANs have been shown to achieve comparable or superior results with significantly fewer parameters compared to MLPs. A key feature is their interpretability, as the learned activation functions can be expressed as explicit mathematical equations.

While KANs show theoretical promise, their practical application to time series forecasting presents several key challenges. First, the spline-based activation functions must effectively capture temporal dependencies across varying time scales. Second, the parameter efficiency advantage of KANs must be balanced against their increased computational cost - training is typically an order of magnitude slower than MLPs due to the inability to leverage batch computation across different activation functions. Third, the interpretability benefits must be rigorously evaluated

in the context of time series, where understanding the learned patterns is particularly valuable for applications like financial forecasting and anomaly detection.

The primary objective of this paper is to conduct a comprehensive and methodologically sound evaluation of KANs in time series forecasting applications. While previous work has shown promising results, we aim to provide a more systematic comparison by ensuring all architectures - both KAN-based and traditional neural networks - have access to equivalent input representations. By testing T-KAN and MT-KAN architectures against properly configured baseline models on financial market data, we can better assess whether KAN's theoretical advantages in parameter efficiency and interpretability translate to practical benefits. This careful empirical investigation will help determine if KANs truly represent a viable alternative to established neural architectures for time series applications, particularly when controlling for input dimensionality and model capacity.

2. Current Approaches and their Limitations

Time series forecasting has witnessed significant advancements through various deep learning architectures. Recent developments include the Channel Aligned Robust Blend Transformer (CARD) [9], which enhances accuracy through channel alignment within transformer models, and TimeMixer [6], which employs decomposable multiscale mixing approaches. Additionally, models like MSGNet [7] leverage multi-scale inter-series correlations for multivariate forecasting, while Crossformer exploits cross-dimensional dependencies.

Despite these advancements, current approaches face several fundamental limitations. Many models struggle with interpretability, functioning as "black boxes" that provide limited insight into their decision-making processes. This opacity is particularly problematic in domains like finance and healthcare, where understanding model behavior is crucial for decision-making. For instance, in financial markets, investors prioritize understanding regime switches and uncovering potential nonlinear relationships among stock series over pure predictive accuracy. Additionally, current methods often struggle with concept drift in time series data.

Given these challenges and the current landscape, our study deliberately evaluates Kolmogorov-Arnold Networks (KANs) against fundamental architectures like MLPs, RNNs, and LSTMs rather than more sophisticated models. This methodological choice is justified by how new KAN is . It establishes clear performance benchmarks against

well-understood models, allows for isolation of KAN's unique architectural benefits, and provides a foundation for future comparisons with more advanced architectures.

KANs, with their learnable activation functions and edge-based computations, present a promising approach to addressing these limitations, particularly in terms of interpretability and concept drift detection. However, their practical advantages require thorough empirical validation through careful comparison with established baseline models before progressing to more sophisticated architectures.

3. Research Significance and Potential Impact

The primary objective of this research is to rigorously examine the capabilities and limitations of KANs in practical applications. By systematically testing KAN architectures in time series forecasting tasks, we seek to establish their viability as potential replacements for MLPs in specific contexts. This evaluation encompasses both theoretical aspects, such as scaling properties and computational efficiency, and practical considerations including interpretability and ease of implementation.

Our work has particular relevance for domains where interpretable models are crucial for decision-making processes. In financial forecasting, for instance, understanding the relationship between input features and predictions is often as important as the accuracy of the predictions themselves. The inherent interpretability of KANs offers a promising direction for such applications.

Furthermore, this research contributes to the broader discussion of architectural innovations in deep learning. By thoroughly documenting both the strengths and limitations of KANs, we provide valuable insights for researchers and practitioners considering alternative approaches to traditional neural network architectures. This includes analysis of computational requirements, scaling behavior, and practical implementation challenges that must be addressed for wider adoption of KAN-based approaches.

The findings from this investigation could inform future developments in neural network architecture design, particularly in applications where interpretability and efficient parameter utilization are paramount. However, it is essential to note that further research is needed to fully understand the scalability and practical limitations of KANs in more complex applications.

4. Approach

4.1. Data Preparation

In our study, we utilized two primary datasets: the daily volatility of the S&P 500 and a comprehensive set of macroeconomic indicators from the Federal Reserve Economic Data (FRED). The S&P 500 daily volatility data serves as our univariate time series, providing high-frequency measurements of market volatility that are crucial for financial risk assessment and market behavior analysis.

The macroeconomic indicators from FRED comprise various economic metrics including GDP, Unemployment Rate, CPI, PPI, Industrial Production Index, Federal Funds Rate, Treasury Yields (10-year and 2-year), Corporate Bond Yields, Retail Sales, Housing Starts, USD/EUR Exchange Rate, Oil Prices, Yield Spreads, VIX, and the Consumer Confidence Index. These indicators were selected based on their significance in economic analysis and their ability to capture diverse aspects of macroeconomic dynamics.

A key consideration in our data preprocessing approach was the handling of different sampling frequencies. While Treasury yield data was available on a daily basis, several macroeconomic indicators like GDP were reported quarterly. We employed an outer join operation followed by forward filling of missing values. This approach preserves the actual reported values and maintains the inherent step-wise nature of less frequently reported economic indicators, which is more appropriate for economic time series where values typically remain constant until the next official release.

These datasets were chosen to evaluate KANs in both univariate and multivariate time series forecasting scenarios, with direct applications to financial and economic forecasting tasks. Both datasets are publicly accessible through standard channels, facilitating reproducibility while adhering to appropriate licensing agreements. This selection of data enables us to assess the effectiveness of KANs against established machine learning models in handling real-world forecasting challenges, particularly in terms of performance, interpretability, and computational efficiency. I'll write a new subsection focusing on the volatility calculation and post-processing steps. Here's my proposed addition:

4.2. Target Variable and Post-Processing

Our primary target variable is the realized volatility of stock returns, which serves as a proxy for actual market volatility. Following standard financial literature, we com-

pute the daily realized volatility using the following formula:

$$V_t = \sqrt{\sum_{t=1}^n (r_t)^2} \quad (4)$$

where $r_t = \ln(c_t/c_{t-1})$ represents the log returns, and c_t is the adjusted closing price at time t . This approach provides a measure of price variability that is widely accepted in financial research and practice. To ensure the validity of our evaluation metrics and the interpretability of our results, we implement standard normalization (z-score normalization) to scale the volatility values. This standardization process transforms the data to have zero mean and unit variance:

$$V_{normalized} = \frac{V_t - \mu}{\sigma} \quad (5)$$

where μ is the mean of the training set volatilities and σ is their standard deviation. For the inverse transformation of our predictions, we apply:

$$V_{denormalized} = V_{normalized} \cdot \sigma + \mu \quad (6)$$

This de-normalization step is crucial for meaningful interpretation of our results and for calculating evaluation metrics in the original scale. These post-processing steps enable us to generate meaningful visualizations of the predicted volatility time series while maintaining the statistical validity of our evaluation metrics. The processed predictions can be directly compared with actual market volatility measures, providing a clear assessment of our model's forecasting capabilities.

4.3. Experimental Setup

We designed our experimental framework to rigorously evaluate KAN's performance against established baselines in time series forecasting. Initial feature selection experiments using a simple RNN architecture revealed that higher granularity features (daily data) provided superior predictive value compared to lower frequency features (quarterly data), informing our final feature selection. We applied standard data normalization techniques to stabilize gradient flow and improve model convergence, with appropriate de-normalization for result visualization and metric computation.

While KAN was not originally designed for sequential data, we adapted it for time series forecasting by reformulating the task as a supervised learning problem. This adaptation follows the T-KAN methodology, where input sequences are constructed using multiple sliding windows. Formally, each input comprises n_{sw} sliding windows, with each window of size w matching the target forecast horizon h .

The KAN training pipeline follows a four-stage process: initialization of the network parameters, structural pruning to remove redundant connections, optional grid refinement for improved accuracy, and symbolic fixing to enhance interpretability. Model fitting is performed after each stage to ensure optimal performance.

For comparative analysis, we implemented three baseline architectures: MLP, RNN, and LSTM. The MLP implementation mirrors KAN’s supervised learning approach, accepting flattened inputs of dimension $n_{sw} \times w$ to generate predictions over horizon h . In contrast, the sequential models (RNN and LSTM) process the input differently, consuming one window of size w per time step across n_{sw} sequential steps, while maintaining the same output dimension h for forecasting.

We established three benchmark periods beginning from 2023-12-01, with forecast horizons $h \in 1, 5, 21$ days, corresponding to daily, weekly, monthly predictions respectively. To ensure methodological consistency, we maintained identical window parameters (n_{sw}, w) across all horizon benchmarks.

All models underwent hyperparameter optimization to achieve satisfactory performance metrics while maintaining reasonable computational costs. The choice of these traditional architectures as baselines, rather than more recent sophisticated models, aligns with the comparative framework suggested by the T-KAN authors, providing a fair evaluation context for the relatively novel KAN architecture.

5. Implementation Challenges and Solutions

In implementing KAN for financial forecasting, we encountered several anticipated and unexpected challenges. This section details these challenges and our approach to resolving them.

5.1. Computational Efficiency

The computational challenges were significant and multifaceted. As documented by Liu et al. [4], KAN’s training time was approximately 10 times slower than comparable MLPs, requiring careful resource allocation. The spline-based activation functions demanded significant memory overhead, necessitating a strategic approach to scaling. We implemented an incremental scaling strategy, starting with a minimal network architecture and gradually increasing network width and depth. Hyperparameters were fine-tuned at each scale before expansion to ensure optimal performance and resource utilization.

5.2. Code Implementation Challenges

Working with early-stage software presented numerous technical hurdles. The pykan package, while publicly available, exhibited several limitations including a critical pruning sequence bug where nodes were being pruned before edges. Rather than downgrading the package, we developed custom solutions including a pruning sequence handler and additional validation checks for model consistency. The implementation of T-KAN and MT-KAN architectures required careful interpretation of the original papers [8], as their code bases were not publicly available at the time of our study.

5.3. Training Stability

Training stability emerged as a critical challenge in our implementation of KAN for financial forecasting. We frequently encountered gradient explosion issues manifesting as NaN values in training loss, particularly during the early stages of implementation.

To address these stability issues, we implemented several key solutions. First, gradient clipping was introduced as a primary stabilization mechanism to prevent extreme gradient values from disrupting the training process. We also increased the overall number of training datapoints to provide more stable gradient estimates during backpropagation. The *avoid_singularity* parameter was enabled to prevent numerical instabilities in the computation of spline functions.

For MT-KAN specifically, we found that updating the grid parameters during training led to instability, so we disabled grid updates in this variant. While we experimented with learning rate decay schedules for MT-KAN, this approach did not yield significant improvements in training stability and was ultimately not included in our final implementation.

The hyperparameter tuning process proved particularly challenging despite having reproducible results through fixed random seeds. Many parameter combinations led to gradient explosion, severely limiting the viable parameter space. This constrained range of usable parameters made the development process considerably more time-consuming than traditional neural networks. Our experience suggests that KAN’s training stability, particularly for financial time series applications, remains an area requiring further research and development. The fundamental challenge appears to lie in balancing the model’s expressive power through its spline-based architecture with achieving consistent training stability.

5.4. Code Base and Modifications

Our implementation required substantial modifications to the base pykan repository [5]. Core modifications included gradient clipping for gradient updates, and custom data loaders for financial time series. We also developed monitoring tools for spline parameter evolution. New features were added to support time series-specific validation metrics, automated hyperparameter logging, and custom visualization tools for spline evolution. Integration with financial data APIs was implemented to streamline data acquisition and preprocessing.

6. Results Analysis

6.1. Evaluation Metrics

To rigorously assess model performance, we employed three standard metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE). Additionally, we tracked the number of parameters to measure model efficiency and training time to evaluate computational cost. These metrics provide a comprehensive Ah, you want to use LaTeX tags for cross-referencing. Here's the revised version with proper LaTeX tags:

6.2. Quantitative Performance Analysis

Our experimental results revealed several interesting findings that both support and challenge the theoretical advantages of KAN architectures (see Tables 2, 3, and 4 in Appendix). For 1-day horizon predictions, MT-KAN achieved the best performance metrics, though this came at the cost of significantly more parameters than initially expected from KAN's theoretical foundation. The 1-week horizon predictions saw RNN models demonstrating superior performance with remarkably few parameters, while for 1-month predictions, traditional MLP models with moderate complexity showed the best results.

6.2.1 Qualitative Analysis of Predictions

Visual inspection of the predicted time series reveals notable differences in prediction patterns across architectures (see Figures 1, 2, 3, 4, 5, 2, 7, and 8 in Appendix). While all models generally captured the main trends, KAN variants exhibited more volatile predictions with frequent oscillations compared to MLP, RNN, and LSTM models. This increased prediction volatility suggests that KAN may be more sensitive to minor variations in the input data, potentially overfitting to noise in the training set. Traditional architectures produced smoother prediction curves that better aligned with the general behavior of financial time series. This observation is particularly interesting given KAN's theoretical advantages in function approximation, and may

indicate a need for additional regularization techniques specific to KAN architectures when applied to time series data.

6.2.2 Parameter Efficiency

One surprising finding that challenges the original KAN paper's claims is that MLPs could achieve comparable or better performance with fewer parameters than T-KAN. For instance, in our 1-day horizon experiments, an MLP with configuration achieved similar accuracy to T-KAN while using only less parameters compared to T-KAN. This observation raises important questions about the theoretical parameter efficiency claims made for KAN architectures.

6.2.3 Training Efficiency

A significant practical limitation we encountered was the training time for KAN variants. Our implementations consistently showed training times more than 100 times longer than equivalent MLP architectures. While this could potentially be addressed through future engineering optimizations, it presents a serious constraint for real-world deployments where rapid model updates are necessary.

6.2.4 Model Performance Across Horizons

Breaking down performance by prediction horizon reveals interesting patterns:

- 1-day horizon: MT-KAN achieved best-in-class performance
- 1-week horizon: RNN models demonstrated superior results while maintaining parameter efficiency
- 1-month horizon: Traditional MLP-M architecture outperformed all

6.3. Interpretability Trade-offs

While KAN architectures provide theoretical advantages in interpretability through their learnable activation functions, we found this benefit diminishes with increased input dimensionality. In our multivariate time series experiments with 100+ variables, the resulting symbolic expressions became too complex for meaningful interpretation, suggesting a practical limit to KAN's interpretability advantages.

6.4. Discussion

These results suggest that while KAN architectures show promise, their practical implementation faces several challenges that weren't apparent in the theoretical framework. The surprisingly strong performance of simpler architectures like MLPs might indicate that our financial time series problems don't require the full representational

power of KAN architectures, or that current KAN implementations haven't fully realized the theoretical advantages of the architecture.

The higher parameter counts and training times of our KAN implementations, compared to the original paper's claims, might stem from differences in implementation details or parameter counting methodologies, though this cannot be definitively confirmed as the original papers don't provide detailed parameter accounting methods.

7. Analysis of Deep Learning Architecture and Methodology

Our time series forecasting problem was structured as a supervised learning task by converting temporal sequences into input-output pairs. This transformation allowed us to leverage KAN and MLP architectures while preserving the temporal dependencies in the data. Specifically, we used sliding windows to create training examples where each input consisted of historical time steps used to predict future values.

The learnable parameters in our KAN implementation were primarily the spline coefficients that defined the activation functions on the edges between nodes. Unlike traditional MLPs which learn linear weights and biases, KAN learns flexible univariate functions parametrized as B-splines. The spline parameters were continuously updated during training to capture complex temporal patterns. The network structure itself (number of layers, nodes per layer) remained fixed and did not have learnable parameters.

For the loss function, we employed Mean Squared Error (MSE) which is well-suited for regression problems like time series forecasting. Early in development, we observed overfitting where the model performed well on training data but failed to generalize. To address this, we expanded our training dataset and introduced regularization through dropout and L1/L2 penalties. The regularization helped constrain the model's complexity and improved generalization performance. The key hyperparameters in our KAN implementation included:

Grid size (G): Controls the granularity of the spline approximation. Larger G allows more accurate function approximation but increases computational cost. Spline order (k): Determines the degree of polynomial pieces in the B-spline (we used $k=3$ for cubic splines). Higher orders enable smoother functions but may be prone to oscillations. Network width: Number of nodes per layer affects model capacity. We found that even with small widths (5 nodes), KAN achieved competitive performance due to its expressive activation functions.

For optimization, we used LBFGS for KAN as it handles the spline parameters effectively, while ADAM was used for the MLP benchmarks due to its robust performance on traditional neural networks. The hyperparameters were tuned progressively, starting with conservative default values and gradually increasing complexity while monitoring validation performance. This systematic approach helped us find a good balance between model expressiveness and computational efficiency.

We observed that grid size had the most significant impact on model performance - too few grid points limited the model's ability to approximate complex functions, while too many led to overfitting and slower training. The spline order mainly affected smoothness of predictions, with cubic splines ($k=3$) providing a good balance. Network width had less impact than in traditional MLPs, as KAN's learnable activation functions provided sufficient expressiveness even with narrow layers.

8. Work Division

Table 1 presents the distribution of work among team members for this project. The table outlines the primary contributions of each team member, along with detailed descriptions of their individual work.

9. Conclusion

Our empirical investigation into Kolmogorov-Arnold Networks (KAN) for time series tasks reveals a more nuanced picture than initially suggested by theoretical frameworks. While KAN demonstrates promising characteristics in interpretability and expressive power, our experiments highlight significant practical limitations. The implementation complexity and slower training times - typically 10 times longer than equivalent MLPs - present substantial barriers to adoption. Moreover, KAN's high expressiveness can lead to over-parameterization, with models sometimes requiring over 100 variables to represent relatively simple relationships, raising questions about its practical utility.

Our benchmarking experiments show that KAN's performance advantages are not as clear-cut as hypothesized, and the additional computational overhead makes it difficult to justify over simpler alternatives in many scenarios. Nevertheless, KAN represents an important theoretical advancement warranting further research, particularly in addressing implementation challenges and optimizing training efficiency while maintaining its unique interpretability advantages.

References

[1] J. Braun and W. Griebel. The kolmogorov-arnold representation theorem and its applications in numerical analysis. *Numerical Algorithms*, 51(3):329–346, 2009. 1

[2] Andrey Kolmogorov. On the representation of continuous functions of several variables as superpositions of continuous functions of one variable and addition. *Doklady Akademii Nauk SSSR*, 114:953–956, 1957. 1

[3] Andrey Kolmogorov. On the representation of continuous functions of several variables by superpositions of continuous functions of one variable and addition. *Soviet Mathematics - Doklady*, 2:953–956, 1961. 1

[4] Ziming Liu. Kan: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, April 2024. Submitted on 30 Apr 2024, last revised 16 Jun 2024. 1, 4

[5] Ziming Liu. pykan: A python implementation of kolmogorov-arnold networks. <https://github.com/KindXiaoming/pykan>, 2024. GitHub repository. 5

[6] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. Timemixer: Decomposable multiscale mixing for time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, Ant Group, Hangzhou, China and Tsinghua University, Beijing, China, 2024. Published as a conference paper at ICLR 2024. 2

[7] Yuankai Wu. Msgnet: Learning multi-scale inter-series correlations for multivariate time series forecasting. *arXiv preprint arXiv:2401.00423*, December 2023. Submitted on 31 Dec 2023. 2

[8] Kunpeng Xu. Kolmogorov-arnold networks for time series: Bridging predictive power and interpretability. *arXiv preprint arXiv:2406.02496*, June 2024. Submitted on 4 Jun 2024. 1, 4

[9] Tian Zhou. Card: Channel aligned robust blend transformer for time series forecasting. *arXiv preprint arXiv:2305.12095*, May 2023. Submitted on 20 May 2023, last revised 16 Feb 2024 (v5). 2

Appendix

Student	Contributed Aspects	Details
Jun Ming, Chen	- Data Collection - Implementation - Experiment Analysis	- Led the data collection process, identifying and curating relevant datasets - Implemented core model architecture and resolved critical bugs in training pipeline - Conducted extensive hyperparameter tuning experiments - Documented experimental results and findings
Zi Yi, Ker	- Data Preparation - Implementation - Experiment Analysis	- Developed data processing pipeline for datasets - Implemented T-KAN and MT-KAN - Set up the model training scripts - Contributed to technical documentation and result visualization

Table 1. Contributions of team members.

Model	MSE	RMSE	MAE	Time (s)	Params Count
MLP	7.98E-06	2.82E-03	2.82E-03	0.03	136
MLP-M	3.60E-06	1.90E-03	1.90E-03	0.12	56024
RNN	2.49E-06	1.58E-03	1.58E-03	0.09	186
RNN-M	7.89E-06	2.81E-03	2.81E-03	16.3	48381
LSTM	1.46E-06	1.21E-03	1.21E-03	0.16	726
LSTM-M	1.01E-05	3.18E-03	3.18E-03	12.5	62946
T-KAN	5.69E-05	7.54E-03	7.54E-03	15.2	3380
MT-KAN	4.00E-07	6.33E-04	6.33E-04	134	52130

Table 2. 1-day Horizon Benchmark Results

Model	MSE	RMSE	MAE	Time (s)	Params Count
MLP	7.97E-06	2.82E-03	2.59E-03	0.03	560
MLP-M	8.33E-06	2.89E-03	2.72E-03	0.04	8435
RNN	2.70E-06	1.64E-03	1.33E-03	0.11	290
RNN-M	1.17E-05	3.42E-03	3.25E-03	0.12	1865
LSTM	1.23E-05	3.51E-03	3.11E-03	0.31	1550
LSTM-M	1.11E-05	3.34E-03	2.75E-03	0.23	7370
T-KAN	4.41E-05	6.64E-03	6.04E-03	54.77	14300
MT-KAN	4.23E-05	6.51E-03	5.61E-03	925.57	219050

Table 3. 1-week Horizon Benchmark Results

Model	MSE	RMSE	MAE	Time(s)	Params Count
MLP	9.75E-06	3.12E-03	2.69E-03	0.04	1706
MLP-M	8.33E-06	2.89E-03	2.37E-03	0.06	25331
RNN	1.09E-05	3.30E-03	2.87E-03	0.12	596
RNN-M	9.64E-06	3.11E-03	2.70E-03	0.16	5321
LSTM	1.11E-05	3.33E-03	2.82E-03	0.23	2006
LSTM-M	1.29E-05	3.59E-03	3.17E-03	0.28	20906
T-KAN	9.45E-06	3.07E-03	2.66E-03	169.82	43680
MT-KAN	1.61E-05	4.01E-03	3.22E-03	913.74	657930

Table 4. 1-month Horizon Benchmark Results

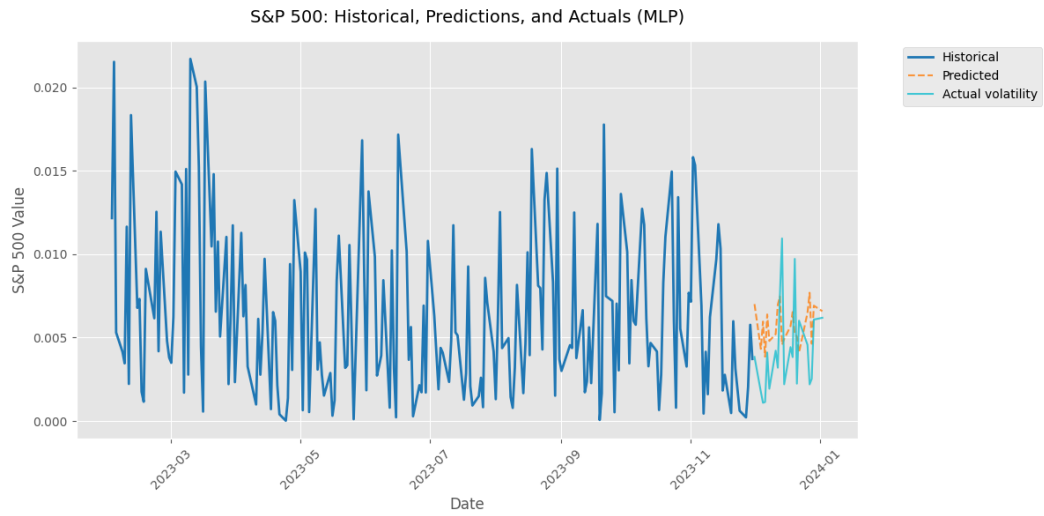


Figure 1. One Month Univariate Forecast of S&P 500 Volatility - MLP

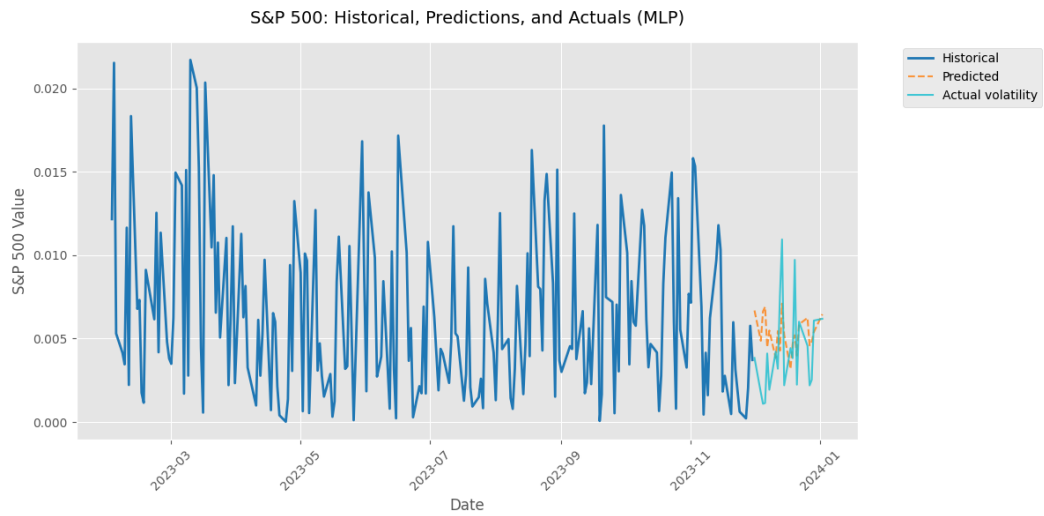


Figure 2. One Month Multivariate Forecast of S&P 500 Volatility - MLP

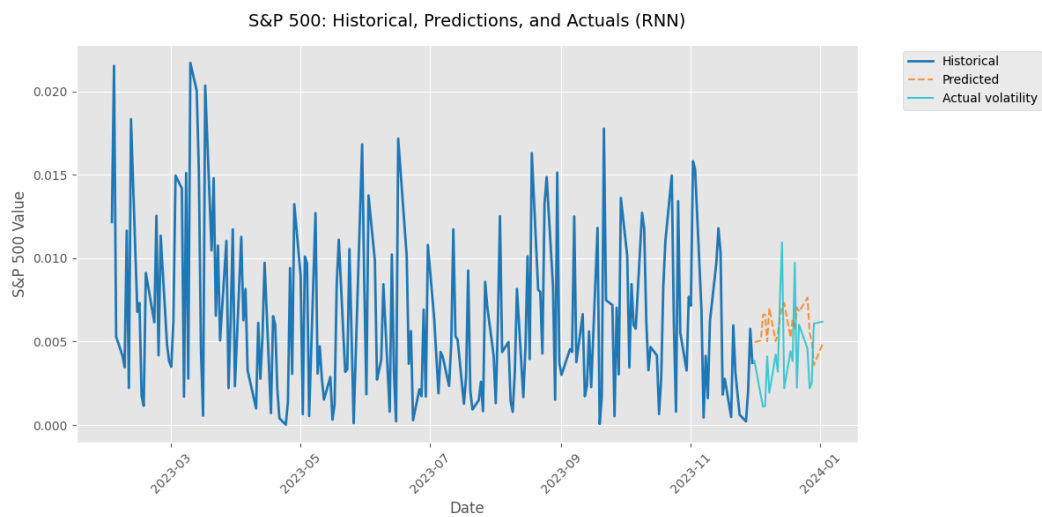


Figure 3. One Month Univariate Forecast of S&P 500 Volatility - RNN

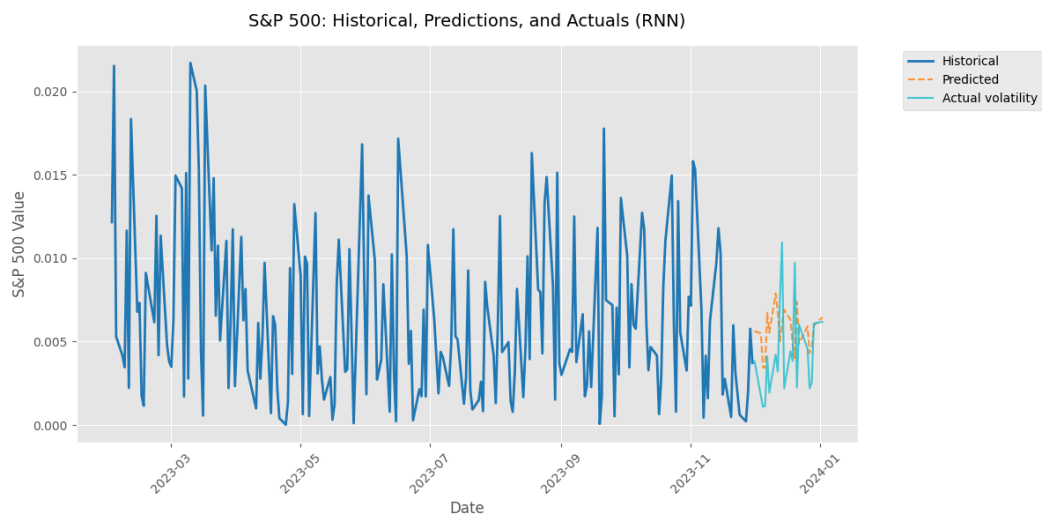


Figure 4. One Month Multivariate Forecast of S&P 500 Volatility - RNN

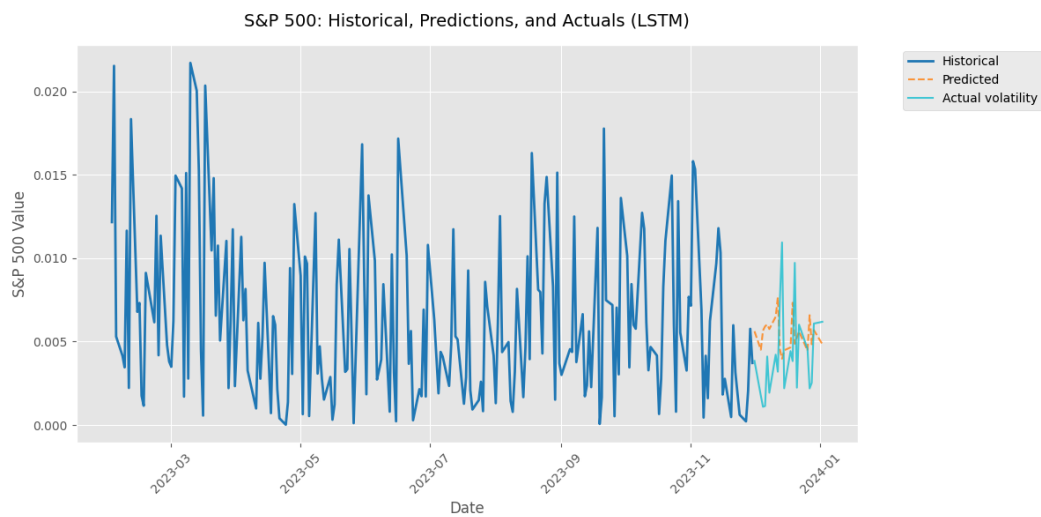


Figure 5. One Month Univariate Forecast of S&P 500 Volatility - LSTM

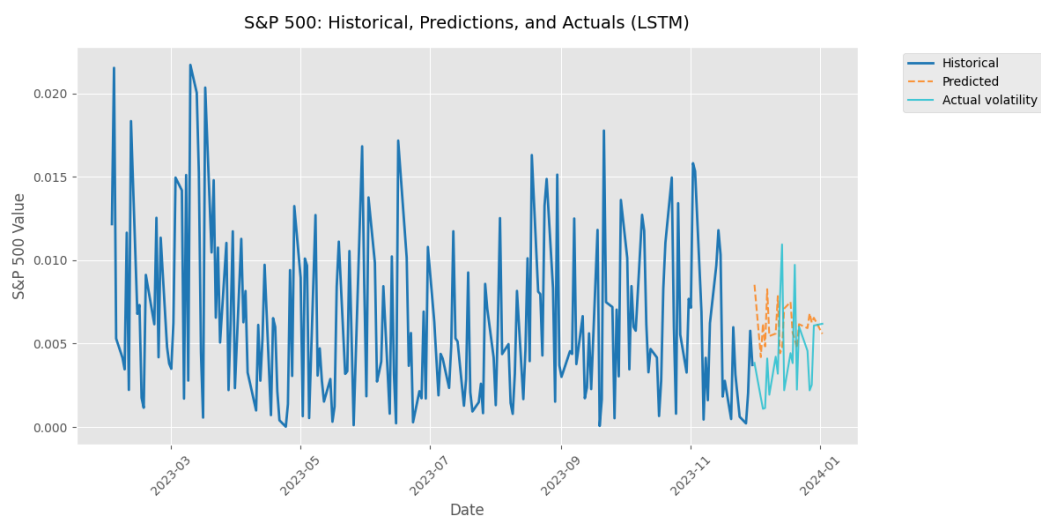


Figure 6. One Month Multivariate Forecast of S&P 500 Volatility - LSTM

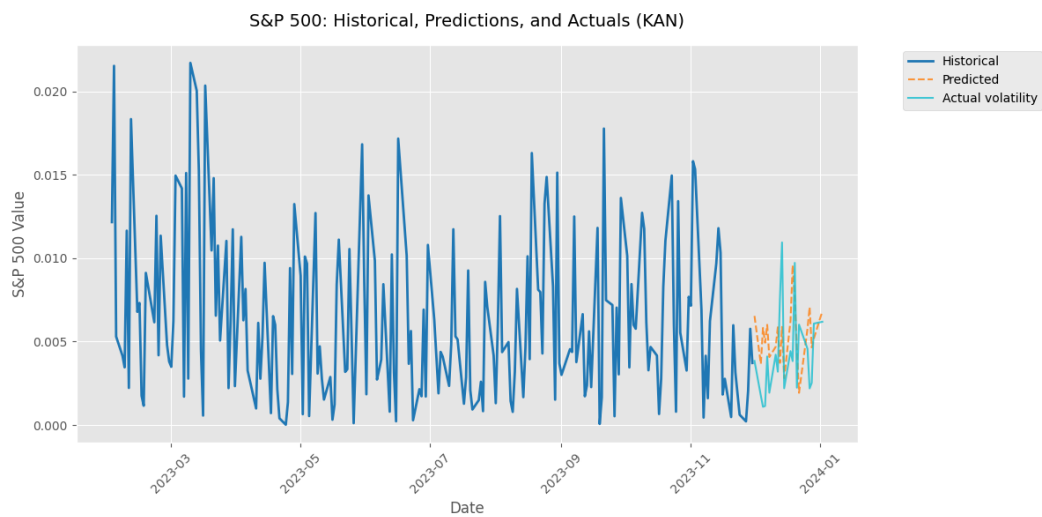


Figure 7. One Month Univariate Forecast of S&P 500 Volatility - KAN

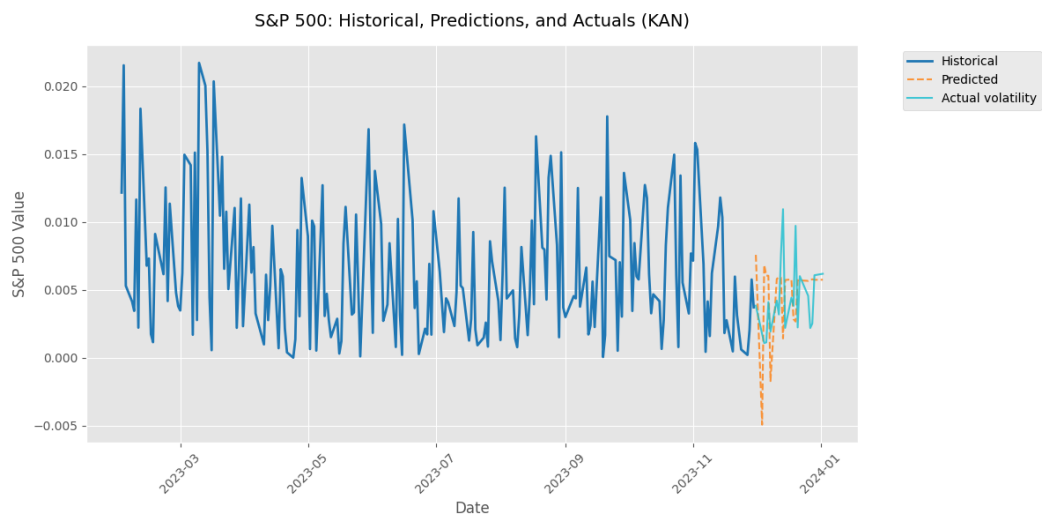


Figure 8. One Month Multivariate Forecast of S&P 500 Volatility - KAN